

Layer	Core tools	Function
Layer 1 — Observability	Prometheus, VictoriaMetrics	Metric collection (15s) and 5-week time-series storage
Layer 2 — Intelligence	scikit-learn, MLflow	Gradient boosting classification + model versioning and audit trail
Layer 3 — Orchestration	Apache Airflow	Weekly DAG execution with human review gate (first 60 days)
Layer 4 — Execution	Kubernetes VPA, CloudNativePG, Patroni	Live resource scaling across pod, operator, and HA topologies

Table 2: Open source tool stack — four-layer framework summary

without interruption, while staging parameters requiring restarts for the next planned maintenance window to protect high-availability integrity.

Real-world impact: Automating the Scale Up/Scale Down decisions

Two PostgreSQL instances drawn from production workloads illustrate the framework's output.

Instance A — reporting database

Week	CPU	Memory	Connections	IOPS
W-4	11%	58%	22%	31%
W-3	9%	61%	19%	28%
W-2	13%	55%	24%	33%
W-1	10%	57%	21%	29%

Weekly metric profile — Instance A (reporting database). All four dimensions remain well within Scale Down territory across W-4 to W-1.

- **Classification: Scale Down:** Every metric falls within Scale Down territory across all four weeks. The model recommends reducing from 8 vCPU/ 32GB to 4 vCPU/16GB. This database was provisioned conservatively at deployment and the workload never expanded — a pattern that would have persisted indefinitely without automated detection.

Instance B — transactional database

Week	CPU	Memory	Connections	IOPS
W-4	61%	71%	74%	68%
W-3	66%	74%	79%	71%
W-2	71%	77%	83%	74%
W-1	74%	81%	89%	78%

Weekly metric profile — Instance B (transactional database). A consistent upward trajectory across all four dimensions signals imminent saturation.

- **Classification: Scale Up:** No individual metric has crossed a critical threshold at any point. A conventional alerting system would not have fired. The model identifies a consistent upward trajectory across all four dimensions simultaneously — a compound pattern that preceded saturation within two to three additional weeks in the training data. Proactive scaling is applied before the incident materialises.

From rules to fingerprints: How the system learns over time

Model accuracy in this framework improves as a function of operational tenure. This compound improvement has three observable phases:

Month one: Recommendations reflect patterns generalised from the initial training corpus. Accuracy is meaningful but imprecise at the instance level.

Month three: With twelve weeks of inference history, the model begins developing behavioural fingerprints — patterns unique to each instance. The reporting database that spikes every Friday at 15:00; the analytics instance that peaks during month-end batch runs. By month three, the model knows that a Monday morning spike on Database is anomalous — because the training data shows Friday is its normal peak.

Month six: Seasonal patterns, sustained growth trajectories, and workload anomalies are fully internalised. The model reliably distinguishes a temporary spike from a structural demand change. Recommendation accuracy at this tenure supports confident transition to full autonomous execution.

This compounding dynamic separates ML-based infrastructure governance from static automation, creating a structured pathway from human-reviewed reports to autonomous resource management.

Beyond PostgreSQL: Adapting the pattern for Redis, Kafka, and Elastic

The 'observe-learn-classify-act' pipeline is infrastructure-agnostic; while specific metrics and scaling endpoints vary by technology, the underlying ML architecture remains identical.

- **Redis:** Uses memory and eviction patterns to scale up before cache hits are impacted or scale down during low use.
- **Kafka:** Monitors consumer lag and replication to predict broker exhaustion, preventing data pipeline outages.
- **Elasticsearch:** Analyses heap and shard patterns to trigger proactive rebalancing or node additions before query latency degrades.

In all scenarios, the Airflow DAG, scikit-learn classifier, and MLflow registry require zero modifications—only Prometheus targets and execution layers change.

Day One deployment: Building your minimum viable pipeline

You don't need to solve the entire problem on Day One. The goal is to make the invisible visible. By the end of the day,